EE 242 Lab 3b – Frequency Domain Representation of Signals - Fourier Transform

Aya Burpee, Bryan Schmid, Tyler Watkins, Arturo Games

This lab has 2 exercises to be completed as a team. Each should be given a separate code cell in your Notebook, followed by a markdown cell with report discussion. Your notebook should start with a markdown title and overview cell, which should be followed by an import cell that has the import statements for all assignments. For this assignment, you will need to import: numpy, the wavfile package from scipy.io, simpleaudio/librosa, and matplotlib.pyplot.

```
# We'll refer to this as the "import cell." Every module you import should be
%matplotlib inline
import numpy as np
import matplotlib
import scipy.signal as sig
import matplotlib.pyplot as plt
# import whatever other modules you use in this lab -- there are more that you
from scipy.io import wavfile
from scipy import signal
from IPython.display import Audio, display
```

Summary

In this lab, we will learn how to transform signals from the time domain to the frequency domain. The concepts we'll focus on include using a discrete implementation of the Fourier Transform (DFT) with a digitized signal and understanding the relationship between the discrete DFT index k and frequency ω for both the original continuous signal x(t). This is a one-week lab.

Lab 3b turn in checklist

• Lab 3b Jupyter notebook with code for the 2 exercises assignment in separate cells. Each assignment cell should contain markdown cells (same as lab overview cells) for the responses to lab report questions. Include your lab members' names at the top of the notebook

the notebook.

Please submit the report as PDF

Assignment 3 -- Analyzing frequency content of a signal

For this assignment, you will use a discrete Fourier transform (specifically, the Python implementation of an FFT) to analyze the frequency content of the 100ms segment of the horn signal (from 200ms to 300ms) from assignment 3A. Because this is a periodic signal, the frequency content will have spikes, but because it is a discrete-time signal, they will have finite height. You will experiment with different FFT sizes and different plotting options. The description below assumes that you import numpy as np.

A. Use the np.fft.fft function to compute the FFT for the 100 ms horn signal, with an fft size of nfft=1024, which you can call **x_f**. Recall that the result of the FFT will be a vector that spans frequencies [0,f_s]. If this is a real-valued signal, then the first half of the FFT matters: [0,nfft/2]. **x_f** is not returned in the desired order so in order to get both the positive and negative frequencies, you need to use the np.fft.fftshift function to get **x_f2**. Create two different plots of the magnitude of result using **(np.abs(.))** in a 2x1 view: one with positive and negative frequencies and one with just positive frequencies. Be sure to scale the magnitude according to time signal window length f_s and signal duration. Label the frequency axis in terms of Hz by creating a vector **freq** that scales the FFT index by f_s/nfft.

B. It is often the case that frequency content is plotted on a log scale. Plot the one-sided (positive frequency) magnitude using using a log scale.

C. Changing the size of the FFT will change the frequency resolution, but it also changes the shape of the result a bit. Just as we saw with Gibbs phenomenon where increasing the number of Fourier series coefficients gave a high frequency ringing at sharp edges, increasing the FFT window will give a “ringing” effect for sharp peaks in frequency. To see this effect, compute the FFT using nfft=2048 and plot the log magnitude only using positive frequencies. Compare to your plot from Part B. The effect is easier to see on a log scale.

```
# Assignment 3 - Analyzing frequency content of a signal

# Part A - Using python FFT function
# Use the np.fft.ftt function to find the fourier domain representation of t
# Use the np.fft.fftshift to find the shifted fft
# Plot and see the positive and negative frequencies (remember to scale the
```

```
# Plot and see the positive and negative frequencies (remember to scale the
    # Seperate out the positive part into another vector
    # Plot the seperated signal
from google.colab import drive
drive.mount('/content/drive/')
fs, horn = wavfile.read("/content/drive/MyDrive/horn11short.wav")
horn = horn[int(fs*0.2):int(fs*0.3)]
nfft = 1024
signal_duration = 0.1
x_f = np.fft.fft(horn,nfft) / (fs*signal_duration)
x_f2 = np.fft.fftshift(x_f) / (fs*signal_duration)


freq_axis1 = np.arange(-nfft/2, nfft/2) * (fs/nfft) # all frequencies
freq_axis2 = np.arange(0, nfft/2) * (fs/nfft) # positive frequencies
# freq_axis2 = np.linspace(0, int(fs/2), int(1/(2*fs/nfft))) # positive freq
# freq_axis1 = np.linspace(-fs/2, fs/2, int(1/(fs/nfft))) # all frequencies
fig, (ax1, ax2) = plt.subplots(2, 1)
fig.subplots_adjust(hspace=0.5)

ax1.plot(freq_axis1, np.abs(x_f2))
ax1.set_title("All Frequencies")
ax1.set_xlabel("Frequency (Hz)")
ax1.set_ylabel("Magnitude")

ax2.plot(freq_axis2, np.abs(x_f2[int(nfft/2):]))
ax2.set_title("Positive Frequencies")
ax2.set_xlabel("Frequency (Hz)")
ax2.set_ylabel("Magnitude")
plt.show()

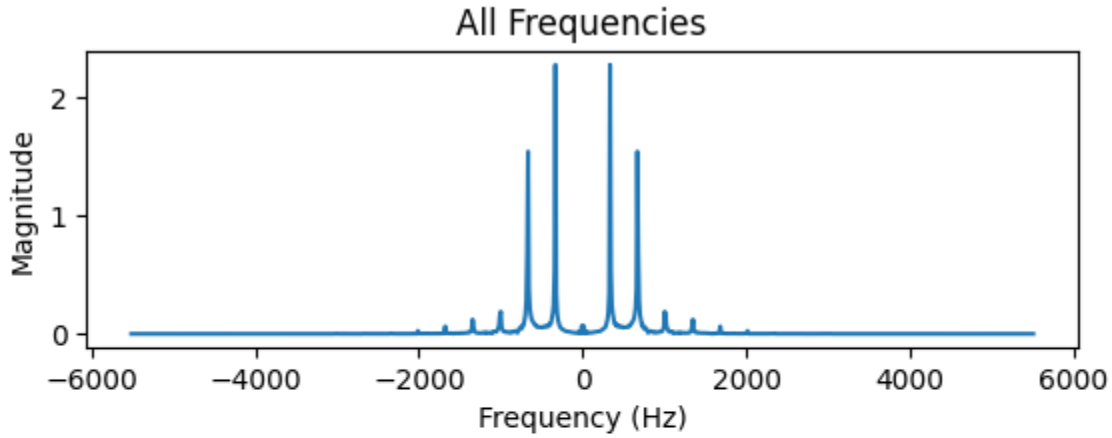
# Part B - Plot the FFT across frequency
# Repeat the process for plotting the positive frequencies
# Use the following to change your x-axis to logscale and plot in parallel -
plt.figure(figsize=(8, 3))
plt.semilogx(freq_axis2, np.abs(x_f2[int(nfft/2):]))
plt.title("Positive Frequencies(log scale), nfft=1024")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.show()

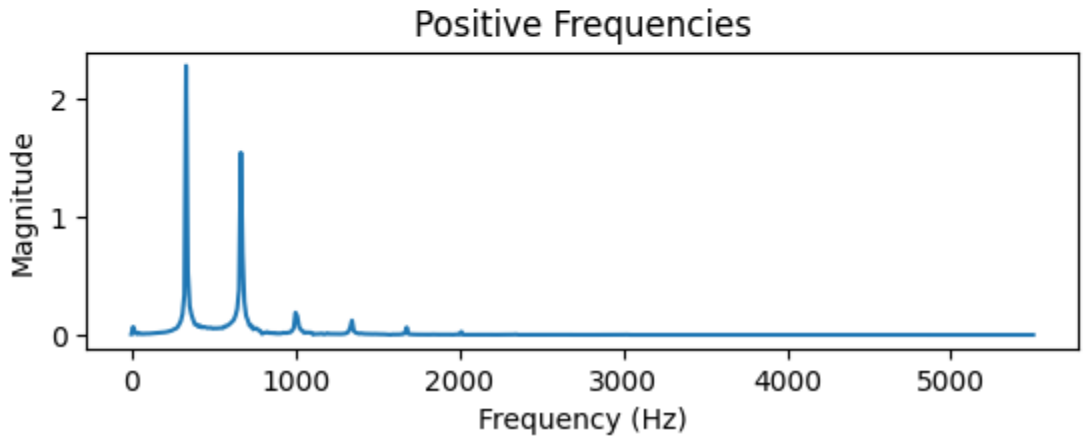
# Part C - Horn Signal Ringing effect.
# Do the same things as Part A but with nfft = 2048
# Plot the the signal (positive frequencies only) and observe the additional
nfft_c = 2048
x_fc = np.fft.fft(horn,nfft_c) / (fs*signal_duration)
x_f2c = np.fft.fftshift(x_fc)
freq_axis3 = np.arange(0, nfft_c/2) * (fs/nfft_c)

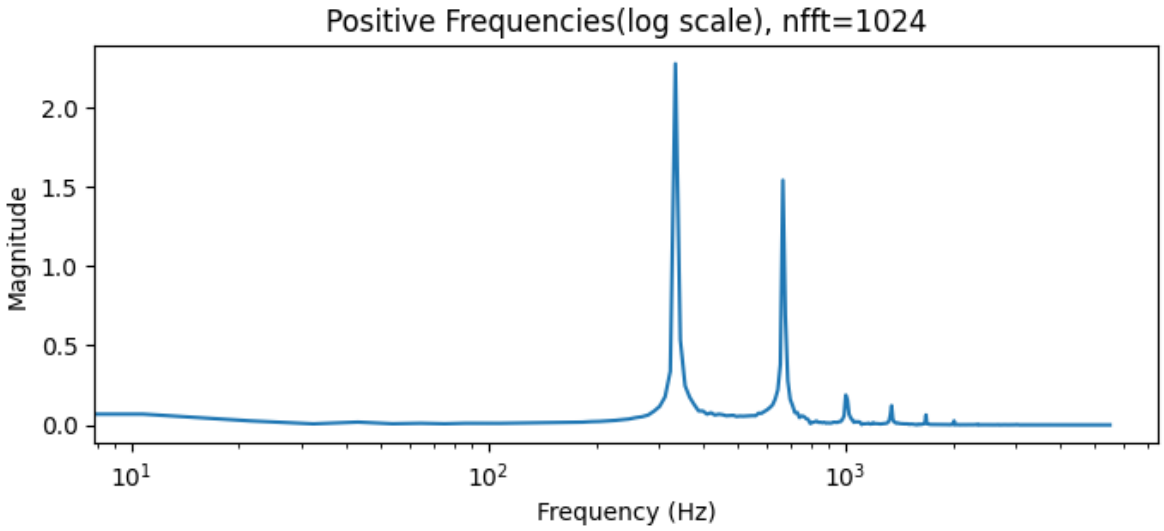
plt.figure(figsize=(8, 3))
plt.plot(freq_axis3, np.abs(x_f2c[int(nfft_c/2):]))
```

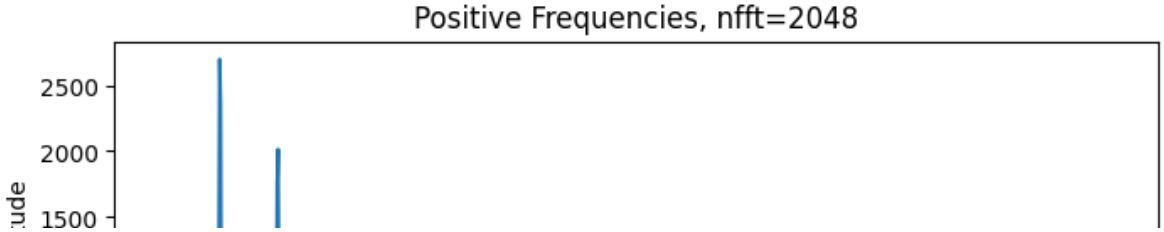
```
plt.title("Positive Frequencies, nfft=2048")
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude")
plt.show()
```

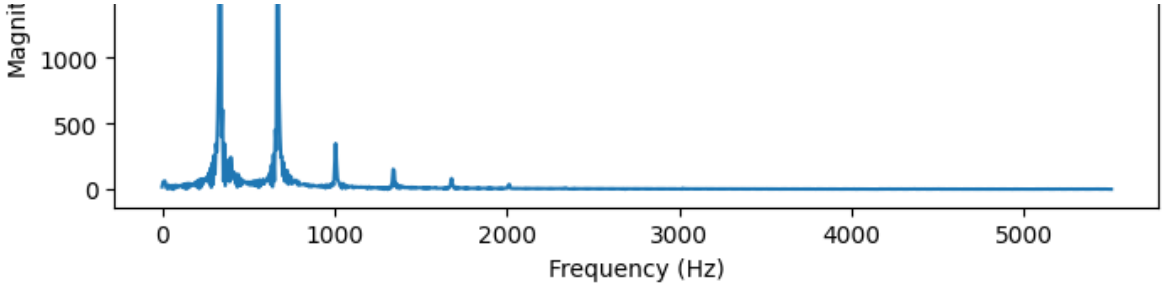
```
Mounted at /content/drive/
```











Discussion

In assignment 3, we used specific cosine frequencies to approximate the horn note, assuming the signal is periodic so the harmonics have non-zero energy. The FFT results show a different picture, and the synthesized version is easily distinguished from the original. Discuss reasons for these differences.

The FFT shows a different picture as the horn note is not perfectly periodic and the fourier transform is doing the best possible to recreate the signal. Also the fundamental frequency will change throughout the note the horn player is playing which will further change the result. The transform cannot perfectly replicate the system through FFT unless it is perfectly periodic.

Assignment 4 -- Comparing frequency content of a signal

Many interesting time signals have changing frequency content. Music is one example, since different notes have different fundamental frequency. Speech is another example: we distinguish different vowels and consonants based on their frequency content. In this assignment, you will use the FFT to compare the frequency content of two different speech sounds in a sentence. We'll use 30ms windows, where the frequency content is relatively stable.

A. Download the signal "bluenose3.wav", and read in the file. Plot the full waveform, using the sampling frequency to correctly label the time axis. Play the file.

B. Extract the samples corresponding to times [0.75,0.78]. (This corresponds to the "oo" sound in the word "grew.") Using a 2x1 plot, plot the time waveform (labeling the time axis with the specified time region) and the magnitude of the frequency response (positive frequencies only, labeling the frequency axis in Hz)

response (positive frequencies only, labeling the frequency axis in Hz).

C. Repeat the exercise above using the samples corresponding to times [2.565,2.595]. (This corresponds to the "s" sound.)

```
nfft = 2048

# Assignment 4 - Comparing frequency content of a signal
# Part A - Reading, plotting and playing the bluenose3.wav file
fs, bluenose = wavfile.read("/content/drive/MyDrive/bluenose3.wav")
x1 = np.linspace(0, len(bluenose)/fs, len(bluenose))
fig, ax1 = plt.subplots()
ax1.plot(x1, bluenose)
ax1.set_title("Signal Magnitude")
ax1.set_xlabel("Time(s)")
ax1.set_ylabel("Magnitude")
display(Audio("/content/drive/MyDrive/bluenose3.wav"))

# Part B - Extracting ooo part of the signal
# Extract the ooo part of the signal from 0.75 seconds to 0.78 seconds
# Plot the signal in time and frequency domain
start1 = int(.75*fs)
start2 = int(.78*fs)

sample = bluenose[start1:start2]
fig, (ax2, ax3) = plt.subplots(2,1)
fig.subplots_adjust(hspace=0.5)
x2 = np.linspace(.75, .78, int(fs*3/100))
ax2.plot(x2, sample)
ax2.set_title("Signal Magnitude")
ax2.set_xlabel("Time(s)")
ax2.set_ylabel("Magnitude")
freq_axis = np.linspace(0, fs/2, nfft//2)
allfreq = np.fft.fft(sample,nfft)
posfreq = np.abs(allfreq[:nfft//2])
ax3.plot(freq_axis, posfreq)
ax3.set_title("Positive Frequencies")
ax3.set_xlabel("Frequency (Hz)")
ax3.set_ylabel("Magnitude")
# Part C - Extracting sss part of the signal
# Extract the sss part of the signal from 2.565 seconds to 2.595 seconds
# Plot the signal in time and frequency domain
start_time = 2.565
end_time = 2.595


start_index = np.where(x1 >= start_time)[0][0]
print(start_index)
end_index = np.where(x1 <= end_time)[0][-1] + 1
print(end_index)
```

```
sample_2 = bluenose[start_index:end_index]
x3 = x1[start_index:end_index]

fig, (ax4, ax5) = plt.subplots(2, 1)
fig.subplots_adjust(hspace=0.5)

ax4.plot(x3, sample_2)
ax4.set_title("Signal Magnitude")
ax4.set_xlabel("Time(s)")
ax4.set_ylabel("Magnitude")

freq_axis_2 = np.linspace(0, fs/2, nfft//2)
xfreq_2 = np.fft.fft(sample_2,nfft)
posfreq_2 = np.abs(xfreq_2[:nfft//2])

ax5.set_title("Positive Frequencies")
ax5.set_xlabel("Frequency (Hz)")
ax5.set_ylabel("Magnitude")

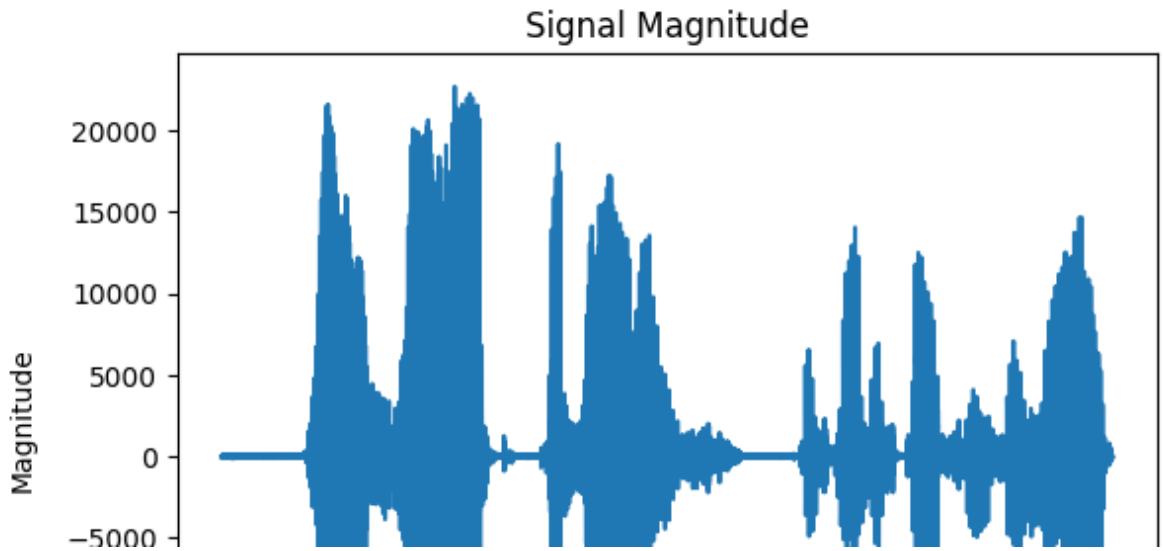
ax5.plot(freq_axis_2, posfreq_2)

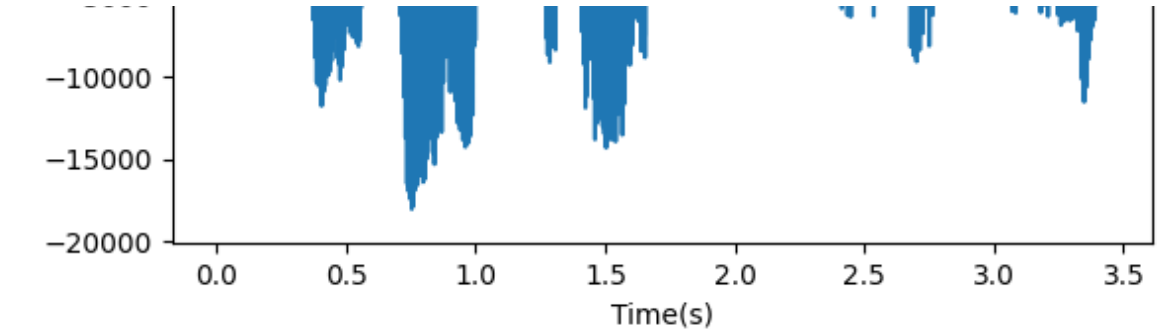

# Part D - Justify the difference and choice of nfft used.

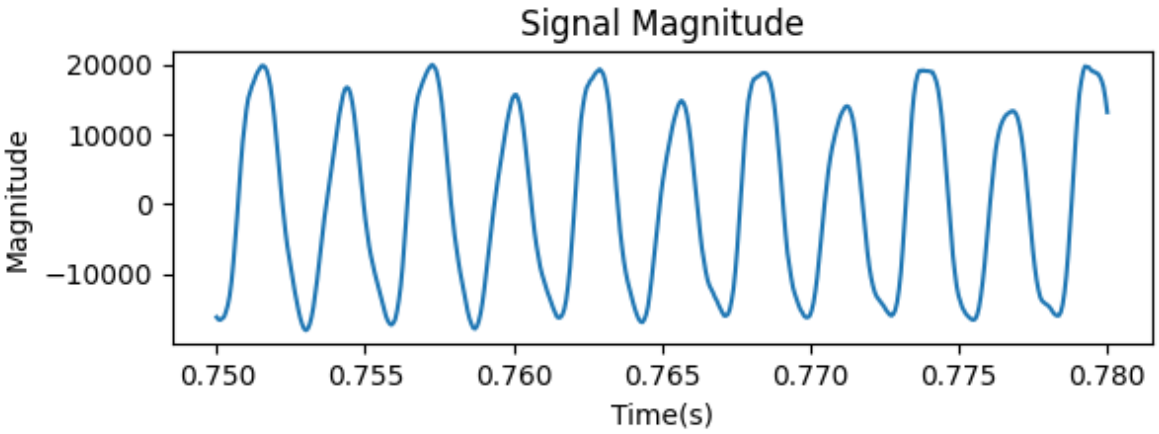
#This is discussed in the discussion below
```

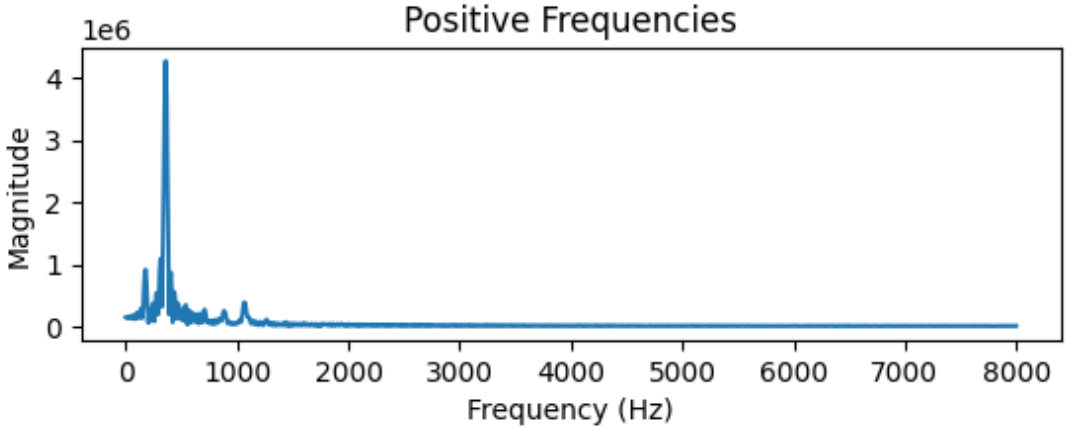
0:00 / 0:03

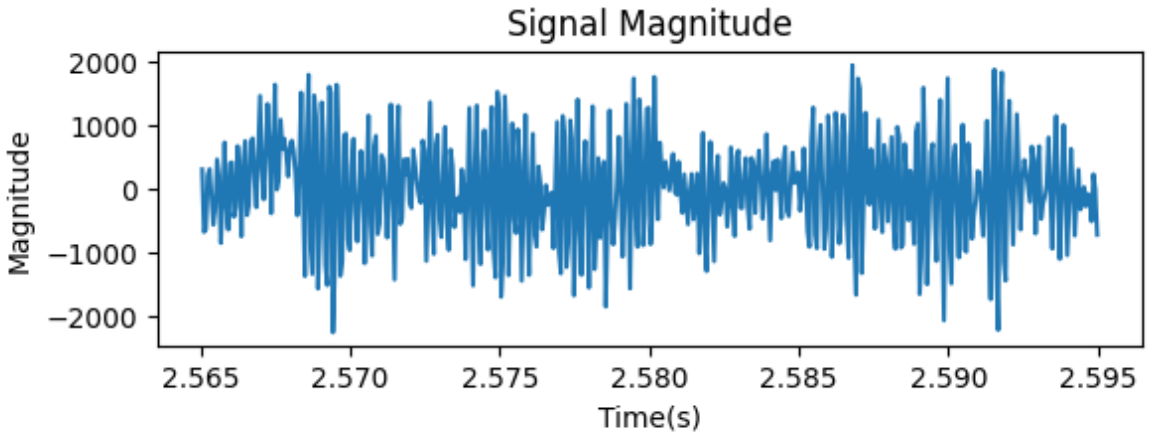
```
41040
41520
[<matplotlib.lines.Line2D at 0x7d85f4176f90>]
```

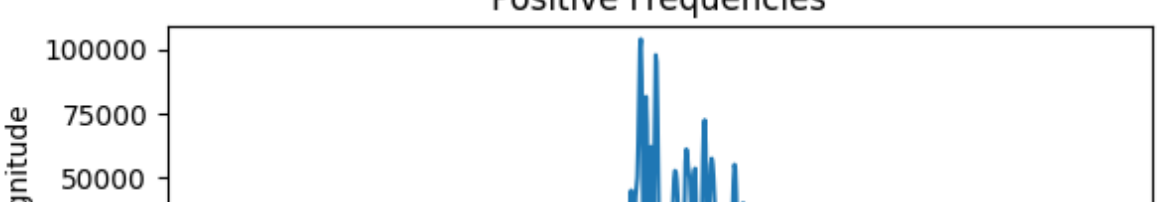


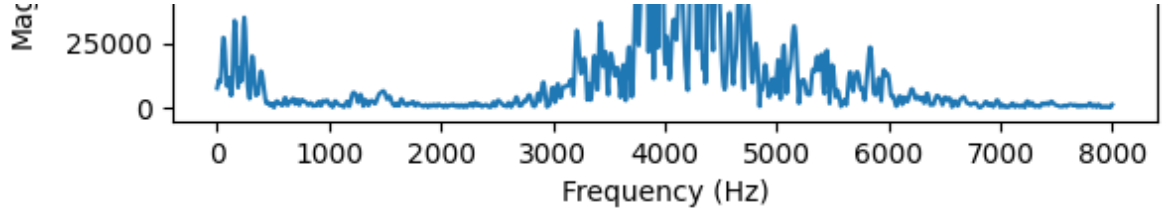

−10000
−15000
−20000
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
Time(s)


Signal Magnitude
Magnitude
20000
10000
0
−10000
0.750
0.755
0.760
0.765
0.770
0.775
0.780
Time(s)


Positive Frequencies
1e6
Magnitude
4
3
2
1
0
0
1000
2000
3000
4000
5000
6000
7000
8000
Frequency (Hz)


Signal Magnitude
Magnitude
2000
1000
0
−1000
−2000
2.565
2.570
2.575
2.580
2.585
2.590
2.595
Time(s)


Positive Frequencies
100000
75000
50000



Discussion

State what size FFT you used and explain your choice. Comment on the differences between the time and frequency plots for the two segments and the auditory differences.

We chose an FFT of 4096 for assignment 4 as the higher value created smaller bins and a more defined peak. This is seen in the plot for part B as a clear peak around 300-400 hertz is present. If a smaller FFT value is used, a less defined peak would be present. Between the two time and frequency plots, the second plot a more random noise and it is apparent in both the frequency graph and time magnitude graph. This is due to the auditory differences of the two sounds, the "sss" sound is much more random in frequency and audio compared to the "ooo".